



Curso:	Disciplina:	Código/Turma:
Professor/a:	Data:	
Aluno/a:	Matrícula:	

Recomendações e regras para a avaliação

- Proibido qualquer **interação com outro estudante** durante o período da prova.
- O tempo de realização da prova será de no máximo 1h40m.
- Os alunos que terminarem a prova antes do final da aula, podem sair de sala após 30 minutos de duração.
- Implemente as questões em Java ou Python e envie o código no AVA

1. DFS Recursiva (5,0 pontos)

Um aplicativo de navegação representa um bairro como um grafo não direcionado, onde cada vértice é um cruzamento e cada aresta é uma rua. Dado um cruzamento de partida e um de destino, o app precisa sugerir **um caminho possível** (não necessariamente o mais curto) e informar **quantas ruas** serão percorridas.

Implemente essa funcionalidade usando **DFS recursiva**, considerando o grafo abaixo, representado por lista de adjacência:

```
0 -> [1, 2, 3]
1 -> [0, 4, 5]
2 -> [0, 5, 6]
3 -> [0, 6, 7]
4 -> [1, 8]
5 -> [1, 2, 9]
6 -> [2, 3, 9, 10]
7 -> [3, 10]
8 -> [4, 11]
9 -> [5, 6, 11, 12]
10 -> [6, 7, 12, 13]
11 -> [8, 9, 14]
12 -> [9, 10, 14]
13 -> [10, 14]
14 -> [11, 12, 13]
```

Origem: 0 Destino: 14

Requisitos:

- Uso obrigatório de **DFS recursiva**.
- Ao escolher entre vizinhos, siga a ordem em que aparecem na lista de adjacência.

- Se não existir caminho, imprima SEM CAMINHO.
- Comente as partes principais (marcação de visitados, chamada recursiva, reconstrução do caminho).

Saída esperada (formato):

```
Caminho: 0 -> ... -> 14
Distância: N
```

2. DFS Iterativa com Pilha (5,0 pontos)

Uma rede de computadores é modelada como um grafo não direcionado, onde cada vértice é uma máquina e cada aresta é um cabo de rede. O administrador quer saber se um pacote pode ser entregue de uma máquina de origem até uma de destino e, se sim, **por qual sequência de máquinas** ele passaria e **quantos saltos** seriam necessários.

Implemente essa funcionalidade usando **DFS iterativa com pilha explícita** (sem recursão), considerando o grafo abaixo, representado por lista de adjacência:

```
0 -> [1, 2]
1 -> [0, 3, 4]
2 -> [0, 4, 5]
3 -> [1, 6]
4 -> [1, 2, 7, 8]
5 -> [2, 8, 9]
6 -> [3, 10]
7 -> [4, 10, 11]
8 -> [4, 5, 11, 12]
9 -> [5, 12]
10 -> [6, 7, 13]
11 -> [7, 8, 13, 14]
12 -> [8, 9, 14, 15]
13 -> [10, 11, 16]
14 -> [11, 12, 16, 17]
15 -> [12, 17]
16 -> [13, 14, 17]
17 -> [14, 15, 16]
```

Origem: 3 Destino: 15

Requisitos:

- Uso obrigatório de **DFS iterativa** com uma **pilha**.
- **Não pode usar recursão**.
- Ao empilhar vizinhos, siga a ordem em que aparecem na lista de adjacência.
- Se não existir caminho, imprima SEM CAMINHO.

- Comente as partes principais (operações de push/pop, marcação de visitados e reconstrução do caminho a partir dos predecessores).

Saída esperada (formato):

Caminho: 3 -> ... -> 15

Distância: N